EVALUACIÓN PARCIAL N°2 · ENCARGO

ANI v8: un agente que acompaña trámites normativos del liceo

Asignatura	ISY0101 Ingeniería de Soluciones con IA · Docente: Cristian Cárcamo M.
Estudiantes	Felipe Villalobos · Adolfo Medina
Repositorio	github.com/felipits/ANI-asistente-normativo (notebook `P6_ANI_agente_v8.ipynb`)
Fecha	Noviembre de 2026

1. Contexto y alcance de la EP2

En la EP1 construimos ANI, un asistente con LLM y RAG (Lewis et al., 2020) que responde dudas de apoderados del Liceo Polivalente Guillermo Labarca Hubertson (Quinta Normal, SLEP Los Parques) sobre sus reglamentos de Evaluación y de Convivencia (Liceo Polivalente Guillermo Labarca Hubertson, 2026) y el Decreto 67/2018 (Biblioteca del Congreso Nacional de Chile, 2018), citando documento, ubicación y página. Esa versión (v7) solo respondía: no dejaba ningún trámite avanzado, perdía la conversación al cerrar Colab y sus decisiones eran de un solo paso.

En la v8 ANI acompaña tres trámites que hoy UTP resuelve en persona: reprogramar una prueba a la que el estudiante faltó, saber si pasa de curso y reclamar cuando las notas no se entregan a tiempo. Para cada uno decide qué corresponde según el reglamento, pide confirmación y deja listo el paso siguiente (registro para la unidad, borrador de solicitud con la norma citada y recordatorio del plazo). Con un código de caso, el apoderado lo retoma otro día.

2. Diseño e implementación del agente (IE1, IE2)

ANI v8 corre en un notebook de Google Colab. El agente usa LangChain (LangChain, s.f.) con `create_tool_calling_agent` + `AgentExecutor` de `langchain-classic`, sobre Gemini (Google, s.f.) a través de `langchain-google-genai` 4.x. Parte con `gemini-3.1-flash-lite` y, si el servicio responde 429 o 503, pasa a `gemini-3.5-flash-lite` y `gemini-3.8-flash`. La interfaz es Gradio (Abid et al., 2019) y tiene dos pestañas que llegan a caminos distintos (Figura 1).

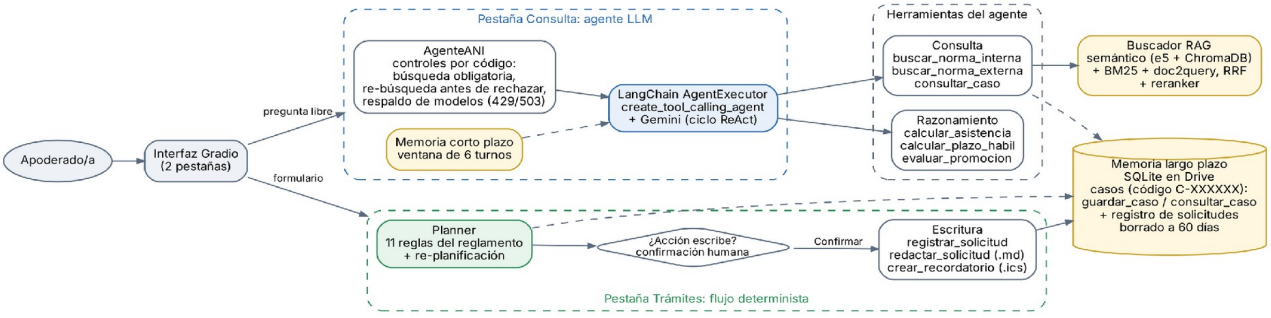


Figura 1. Orquestación de componentes de ANI v8. Elaboración propia.

Consulta (agente LLM). El apoderado escribe una pregunta libre. `AgenteANI` arma el prompt con las 13 reglas de la v7 (`ChatPromptTemplate` con `chat_history` y `agent_scratchpad`) y el `AgentExecutor` repite el ciclo ReAct (Yao et al., 2023): el modelo elige una herramienta, LangChain la ejecuta y le devuelve el resultado, hasta que responde (máximo 8 iteraciones). Alrededor del ejecutor quedan los controles por código de la v7: si el modelo contesta sin buscar se le exige buscar, antes de responder "no está regulado" se hace una segunda búsqueda con sinónimos, y ante saturación se reintenta hasta 3 rondas con 10, 20 y 30 segundos de espera.

Trámites (flujo determinista). El apoderado elige su situación en un formulario que muestra solo los campos de ese trámite. El planner decide la acción (sección 4) y la escritura ocurre únicamente después de **Confirmar y registrar**.

Tipo	Herramienta	Qué hace
Consulta	`buscar_norma_interna`	Busca en los reglamentos y devuelve fragmentos con documento, ubicación y página
Consulta	`buscar_norma_externa`	Busca en el Decreto 67/2018
Consulta	`consultar_caso`	Recupera un trámite guardado a partir de su código (ej. C-4F2A9B)
Razonamiento	`calcular_asistencia`	Calcula el porcentaje de asistencia
Razonamiento	`calcular_plazo_habil`	Cuenta días hábiles descontando fines de semana y feriados de Chile (librería `holidays`)
Razonamiento	`evaluar_promocion`	Art. 10: 1 reprobada exige promedio 4,5; 2 reprobadas, 5,0; asistencia mínima 85 %
Escritura	`registrar_solicitud`	Registra la solicitud para UTP, Dirección o Convivencia (ID `ANI-2026-0001`)

Tipo	Herramienta	Qué hace
Escritura	`redactar_solicitud`	Borrador en Markdown con la norma citada y campos para completar fuera de ANI
Escritura	`crear_recordatorio`	Genera un archivo `.ics` con el plazo, que se abre en cualquier calendario

El agente LLM recibe las seis herramientas de consulta y razonamiento. Las tres de escritura no se le entregan: las llama el flujo de trámites, y rechazan cualquier texto con RUT, correo o teléfono.

3. Memoria y recuperación de contexto (IE3, IE4)

Capa	Implementación	Qué guarda	Duración
Corto plazo	Lista de `HumanMessage`/`AIMessage` en `AgenteANI`; el ejecutor recibe los últimos 6 turnos en `chat_history`	Pregunta y respuesta final, una sola vez por turno	La sesión
Largo plazo	SQLite `ani_memoria.db` en la carpeta ANI de Drive; tablas `casos` y `solicitudes`. La llave es un código de caso, sin login ni RUT	Trámite, acción, razón y plazo; nunca texto libre del apoderado	60 días sin movimiento (`purgar_casos`, Ley 21.719)
Semántica	Índice RAG: e5 multilingüe (Wang et al., 2024) + ChromaDB, BM25 (Robertson y Zaragoza, 2009), 3.057 preguntas hipotéticas (Nogueira et al., 2019), fusión RRF (Cormack et al., 2009) y reranker mMARCO (Bonifacio et al., 2021)	1.142 fragmentos con su página exacta	Permanente; se reindexa si cambia un reglamento

La continuidad funciona en dos escalas: dentro de una conversación, el agente entiende un seguimiento como "¿y si fueron 160 días?"; entre días, el código de caso permite retomar el trámite con **Retomar caso** o escribiéndolo en el chat, donde el agente llama a `consultar_caso`. Los reglamentos pueden leerse en Markdown con marcas `<!-- página N -->`, que conservan la página exacta para la cita.

4. Planificación y toma de decisiones (IE5)

El planner (`decidir_accion`) recibe el estado del trámite y revisa 11 reglas en orden de prioridad. Se detiene en la primera que se cumple. Las que generan escritura quedan a la espera de confirmación.

#	Condición	Acción	Escribe	Fuente
1	Es una situación de convivencia	Deriva a Convivencia Escolar	Sí	—
2	No corresponde a un trámite	Lo responde el agente en el chat	No	—
3	Faltan datos	Pide los datos que faltan	No	—
4	Faltó a una prueba sin justificación	Explica que se exige justificación	No	Eval. Art. 5
5	Justificó, pero pasaron más de 5 días hábiles	Deriva a UTP (fuera de plazo)	Sí	Conv. p. 23
6	Justificó dentro del plazo	Prepara la solicitud de reprogramación	Sí	Eval. Art. 5
7	Cumple notas y asistencia	Informa que es promovido	No	Eval. Art. 10
8	No cumple por notas	Deriva a análisis de UTP	Sí	Art. 10 y 11
9	No cumple solo por asistencia	Prepara la solicitud a Dirección y UTP	Sí	Art. 10, 2d
10	Más de 10 días hábiles sin notas	Prepara el reclamo a UTP	Sí	Eval. p. 9
11	Notas dentro del plazo	Informa cuántos días hábiles van	No	Eval. p. 9

Re-planificación. Cada vez que cambia un dato el planner vuelve a evaluar todo, y `replanificar()` deja registrado el cambio de acción con su motivo; la interfaz lo muestra como "re-planificó: de → a". Dos casos de las pruebas muestran cómo cambia el comportamiento:

- Promoción: con 150 de 180 días (83,3 %) la acción es `solicitar_promocion_excepcional` y pide confirmación. Si el apoderado corrige a 160 días (88,9 %), ANI re-planifica a `informar_promocion` y ya no escribe nada.
- Plazo de notas: una prueba del 1 de octubre sin notas al 4 de noviembre supera los 10 días hábiles y genera `reclamar_plazo_notas`.

Ejemplo real (6 de noviembre). Una inasistencia justificada del 04-11 llevó a `solicitar_reprogramacion` (plazo 11-11); al confirmar se registró `ANI-2026-0002` con borrador, `.ics` y caso `C-4C70FE`. Al corregir la fecha al 13-10, ANI re-planificó a `derivar_utp_fuera_de_plazo` (18 días hábiles) y volvió a pedir confirmación. En Consulta, la adaptación ocurre dentro del agente: elige la fuente, re-busca antes de rechazar y cambia de modelo si el servicio se satura.

5. Pruebas y resultados

Pruebas automáticas (celda 6b). Son 11 pruebas que no gastan cuota de Gemini (base SQLite temporal, fechas fijas y un modelo simulado para la memoria). Cubren el orden de las reglas, las dos re-planificaciones, los feriados, que no se escriba sin confirmación, que con confirmación se generen los 3 archivos y el caso, el rechazo de un RUT y la ventana de memoria (6 turnos, cada turno guardado una vez). Resultado: 11/11. En la prueba manual, el agente respondió "¿Y si fueron 160 días?" con el contexto anterior (88,9 %).

Regresión v7 → v8. Repetimos la evaluación de la EP1 con las mismas 40 preguntas (32 con respuesta en los documentos y 8 que deben rechazarse) y un LLM juez a temperatura 0, para comprobar que la migración a LangChain no bajara la calidad.

Métrica	Umbral	v7 (EP1)	v8 (EP2)	¿Cumple v8?
Exactitud vs referencia (%)	80	92,5	97,5	Sí
Fidelidad / Relevancia (/10)	8	9,8 / 9,7	9,8 / 9,8	Sí
Context Recall (%)	80	93,8	96,9	Sí
Hit@k página esperada (%)	80	96,9	96,9	Sí
Rechazo correcto (%)	100	100	100	Sí
Cita correcta (%)	90	96,9 (31/32)	93,8 (30/32)	Sí
Context Precision (%)	50	27,7	31,7	No
Latencia mediana (s)	—	19,2 (GPU T4)	28,4 (sin GPU)	—

La v8 cumple 6 de 7 criterios. Context Precision sigue bajo el umbral porque el buscador entrega varios fragmentos para no perder el dato. Quedan tres casos por revisar: #28 (rechaza bien, pero agrega una sugerencia), #37 (no llega el Art. 3 del Decreto 67 y cita un artículo inexistente) y #23 (cita equivocada). La latencia no es comparable: la v8 se midió sin GPU.

6. Documentación y repositorio (IE6)

El repositorio `felipits/ANI-asistente-normativo` contiene los notebooks de cada iteración (P1 a P6), los documentos fuente en `ANI/`, los resultados en `evaluacion/`, la bitácora en `docs/BITACORA.md` y la evidencia en `evidencia/` (salida de las pruebas y capturas de la interfaz). El README explica cómo ejecutar ANI en Colab (GPU T4, clave `GEMINI_API_KEY` en Secretos y carpeta `ANI` en Drive) e incluye el diagrama de orquestación y ejemplos de flujo.

7. Decisiones de diseño

a) Diseño híbrido: reglas para los trámites y LLM para las preguntas

ANI tiene dos caminos. La pestaña Consulta usa el agente con LLM y la pestaña Trámites usa un planner de 11 reglas en Python. Se podría haber dejado que el LLM decidiera también los trámites, por ejemplo si un estudiante pasa de curso o si una evaluación se puede reprogramar. No lo hicimos porque el LLM no es determinista: con los mismos datos puede contestar distinto y no hay forma de probar que siempre aplica bien el Art. 10. Una decisión sobre promoción o plazos tiene que salir igual siempre y quedar registrada.

Según el marco visto en clases, este es justo el caso de “no usar agente”: lógica determinista, reglas rígidas y poca tolerancia al error. Las preguntas abiertas son lo contrario. El apoderado dice “se enfermó y faltó” y el reglamento habla de “reprogramación de evaluaciones calificadas”. Esa distancia de lenguaje la resuelve el LLM con RAG, y una regla no puede.

El proyecto lo respalda. Las 11 pruebas de la 6b solo se pueden hacer porque el planner es determinista, y además no gastan cuota. En la prueba rápida, Gemini respondió 503 y 429, y la pestaña Trámites siguió funcionando igual porque no depende del LLM. El costo es que se pierde flexibilidad: hoy hay 3 trámites y agregar uno nuevo significa programar sus reglas. Comparado con TechStore, donde el LLM es casi accesorio porque decide con datos de una tabla, en ANI el centro sigue siendo el buscador con LLM y las reglas se usan solo donde el dato es exacto, como fechas, días y notas.

b) El agente no recibe las herramientas de escritura y todo pasa por confirmación

El agente tiene 6 herramientas de lectura y cálculo. Las 3 de escritura las llama solo el flujo de trámites, después de que el apoderado presiona Confirmar. La diferencia clave es que leer no cambia nada en el mundo y escribir sí: una solicitud registrada para UTP es una acción con efectos.

Si el LLM tuviera acceso a registrar_solicitud, podría llamarla sin que nadie la pidiera, ya sea por un error del modelo o porque alguien escribe en el chat algo como “registra una solicitud de…”, que funciona como inyección de instrucciones. En el dial de autonomía, la escritura queda en N1: ANI propone y un humano aprueba. La regla vista en clases era que más reversibilidad y

menor alcance del daño permiten más autonomía. Un registro dirigido a UTP no es fácil de deshacer, así que le corresponde poca autonomía, mientras que leer sí puede ser autónomo.

Además hay una capa extra: las 3 herramientas rechazan RUT, correo y teléfono, y el resumen se arma solo con campos del formulario, así que no se puede colar un dato personal aunque alguien lo intente. La evidencia está en test_no_escribe_sin_confirmacion y test_escritura_rechaza_rut.

c) Ventana de 6 turnos en vez de ConversationSummaryMemory

El cuaderno 2 de IL2.2 muestra la memoria por resumen, donde el LLM va resumiendo la conversación. No la usamos por tres razones. Cuesta una llamada extra al modelo por turno, y la cuota ya estaba justa, como mostraron los 429. Un resumen puede perder un dato exacto, como "150 días" o una fecha, y en ANI esos son justo los datos que importan. Y las conversaciones con apoderados son cortas y encadenadas ("¿y si…?"), así que 6 turnos completos alcanzan.

Tampoco usamos una clase de memoria automática, porque los controles reintentan la pregunta (búsqueda forzada, re-búsqueda) y una memoria automática guardaría el mismo turno 2 o 3 veces. Por eso el turno se guarda una sola vez, al final. El costo es que después del sexto turno el agente olvida, y lo que tiene que durar más lo cubre la memoria de largo plazo con el código de caso. El seguimiento "¿Y si fueron 160 días?" se respondió bien, y test_memoria_corto_plazo_ventana comprueba el corte.

d) SQLite en Drive con código de caso, sin login ni RUT y con borrado a los 60 días

Elegimos SQLite porque el presupuesto es cero, que es una restricción de la EP1. SQLite es un archivo, no necesita servidor y tiene la forma de un registro real: recibe datos y devuelve un ID, igual que el CRM simulado de TechStore. Va en Drive porque /content se borra al cerrar Colab, y si la memoria de largo plazo viviera ahí no duraría más que la sesión.

Usamos un código en vez de login o RUT por minimización de datos. Son datos de menores y la Ley 21.719 entra en vigencia el 1 de diciembre de 2026. El código no identifica a nadie, y en la base solo se guarda lo que decidió el planner, nunca texto libre del apoderado. Los 60 días responden a que la ley pide no guardar datos más tiempo del necesario, y un trámite de este tipo se resuelve en semanas.

Esto tiene costos. Si el apoderado pierde el código, pierde el caso. Cualquiera que tenga el código puede ver el caso, aunque no contiene datos personales. Y SQLite no está hecho para muchos usuarios a la vez, así que en producción iría una base de datos real o el sistema del liceo.

e) Formulario en vez de sacar los datos del chat

La alternativa era que el LLM extrajera los datos de un mensaje como "mi hijo faltó el lunes a la prueba de mate y tiene licencia". No lo hicimos porque el planner necesita datos exactos (fecha, número de días, cantidad de reprobadas), y extraerlos con el LLM agrega un punto de error justo donde la decisión tiene que ser confiable: ¿qué lunes?, ¿"licencia" cuenta como justificación? El formulario valida tipos, como números y fechas, y pide solo los campos de ese trámite. El costo es que la experiencia es menos natural que conversar, pero el chat sigue disponible para todas las dudas.

f) langchain-classic y langchain-google-genai 4.x en vez de las versiones de la guía

Esto no fue preferencia sino compatibilidad. La guía TechStore fija langchain<1.0 y langchain-google-genai<3. El problema es que cuando un modelo Gemini 3 llama a una herramienta, en el paso siguiente exige que le devuelvan una "firma de pensamiento" (thought_signature). La versión 2.x de langchain-google-genai no la maneja y Gemini responde con error 400, mientras que la 4.x sí la maneja.

Usamos langchain_classic porque ahí quedó AgentExecutor desde LangChain 1.0, y es lo mismo que importan los cuadernos de RA2 del curso, así que el código queda igual al que vimos en clases.

8. Declaración de uso de inteligencia artificial

Usamos Claude (Anthropic, 2026) como apoyo para analizar la pauta y el material de RA2, proponer la estructura de las celdas 5a, 5b, 6b y 7 y escribir su código base, detectar un error en la etiqueta de los protocolos, reforzar la prueba de memoria, generar el diagrama de la Figura 1, redactar las secciones descriptivas 1 a 6 y el README, y dar formato al informe. El equipo revisó todo el código, lo ejecutó en Colab y lo validó con las pruebas automáticas y la regresión de 40 preguntas. Gemini (Google) es un componente de ANI (genera las respuestas y las preguntas hipotéticas, y actúa como juez de la evaluación); no se usó para redactar el informe. Las secciones 7 y 9 y las reflexiones individuales las escribimos sin IA; en la sección 9, Claude solo verificó que los datos coincidieran con los resultados.

9. Conclusiones y reflexiones individuales

Conclusión del equipo

En esta entrega logramos mejorar nuestro asistente normativo institucional ANI, incorporando LangChain con seis herramientas de consulta y razonamiento y tres de escritura, estas últimas con confirmación del usuario. También agregamos una memoria de los últimos seis turnos, almacenamiento de casos mediante SQLite en Drive y un planner de 11 reglas que permite gestionar trámites y adaptarse a cambios en la información.

Además, actualizamos el README, los diagramas y las pruebas del proyecto. En la evaluación de 40 preguntas alcanzamos una exactitud del 97,5 %, lo que demuestra un buen funcionamiento general del asistente.

De todas formas, todavía existen aspectos por mejorar, como la aparición de marcas [F1] en algunas respuestas, el Context Precision de 31,7 % frente al 50 % esperado y la dependencia de las cuotas de Gemini. Como próximos pasos, proponemos mejorar la recuperación de información, optimizar los tiempos de respuesta y validar ANI junto a los equipos de UTP y Convivencia Escolar para avanzar hacia una implementación real en la institución.

Reflexión individual · Felipe Villalobos

Durante esta entrega, mi principal aporte fue revisar y probar los avances del proyecto, verificando que las nuevas funcionalidades funcionaran correctamente. También realicé pruebas en Colab con GPU y participé en la validación del asistente de principio a fin, incluyendo consultas, confirmación de trámites y recuperación de casos. Además, colaboré en la elaboración del informe, la presentación y la organización del repositorio. Como aprendizaje, destaco la importancia de probar cada funcionalidad en situaciones reales y comprobar que los cambios realmente mejoren el funcionamiento del sistema.

Reflexión individual · Adolfo Medina

Durante esta entrega, mi principal aporte fue desarrollar la nueva versión del asistente, incorporando LangChain, el planner, la memoria y sus respectivas funcionalidades. También realicé las pruebas de regresión con 40 preguntas, lo que permitió identificar errores y evaluar el comportamiento del sistema. Además, participé en la documentación del proyecto y en la preparación del informe y el guion. Como aprendizaje, destaco la importancia de separar las reglas de negocio de las consultas al modelo y de validar cada cambio mediante pruebas para evitar nuevos errores.

Referencias

Abid, A., Abdalla, A., Abid, A., Khan, D., Alfozan, A., & Zou, J. (2019). *Gradio: Hassle-free sharing and testing of ML models in the wild*. arXiv. https://arxiv.org/abs/1906.02569

Anthropic. (2026). *Claude* (versión Opus 5.5) [Modelo de lenguaje de gran tamaño]. https://claude.ai

Biblioteca del Congreso Nacional de Chile. (2018). *Decreto 67: Aprueba normas mínimas nacionales sobre evaluación, calificación y promoción escolar*. https://www.bcn.cl/leychile/navegar?idNorma=1127255

Biblioteca del Congreso Nacional de Chile. (2024). *Ley 21.719: Regula la protección y el tratamiento de los datos personales*. https://www.bcn.cl/leychile/navegar?idNorma=1209272

Bonifacio, L., Jeronymo, V., Abonizio, H. Q., Campiotti, I., Fadaee, M., Lotufo, R., & Nogueira, R. (2021). *mMARCO: A multilingual version of the MS MARCO passage ranking dataset*. arXiv. https://arxiv.org/abs/2108.13897

Cormack, G. V., Clarke, C. L. A., & Büttcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. En *Proceedings of the 32nd International ACM SIGIR Conference* (pp. 758–759). ACM.

Google. (s.f.). *Gemini API documentation*. Recuperado el 5 de noviembre de 2026, de https://ai.google.dev/gemini-api/docs

LangChain. (s.f.). *LangChain Python documentation: Agents and tools*. Recuperado el 5 de noviembre de 2026, de https://python.langchain.com

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*, 9459–9474.

Liceo Polivalente Guillermo Labarca Hubertson. (2026). *Reglamento de Evaluación y Promoción 2026* y *Reglamento Interno de Convivencia Escolar 2026* [Documentos institucionales].

Nogueira, R., Yang, W., Lin, J., & Cho, K. (2019). *Document expansion by query prediction*. arXiv. https://arxiv.org/abs/1904.08375

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, *3*(4), 333–389.

Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., & Wei, F. (2024). *Multilingual E5 text embeddings: A technical report*. arXiv. https://arxiv.org/abs/2402.05672

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. En *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2210.03629